Ons Ansible plan:

Good decision.

We'll do this properly — **Ansible competency model**, like Docker and Kubernetes.

Goal:

- Not become Ansible admin.
- Not spend 2 months.
- Become interview-ready.
- Be able to use it in real DevOps projects.
- Understand where it fits in modern stack.

🚀 First — What Is Ansible?

Ansible is:

- Agentless configuration management tool
- Uses SSH
- Declarative (mostly)
- YAML-based automation

It is used to:

- Configure servers
- Install packages
- Deploy middleware (Tomcat, Nginx, etc.)
- Manage users
- Patch systems
- Automate repetitive infra tasks

🧠 Where Ansible Fits in DevOps

Let's compare:

Docker → Packages application
Kubernetes → Orchestrates containers
Terraform → Creates infrastructure
Ansible → Configures servers

In real world:

Terraform → create EC2
Ansible → install Docker on EC2
Docker → build app
Kubernetes → orchestrate app

That's the ecosystem.

🏗️ PHASE 1 — Core Fundamentals (Must Know)

Before writing playbooks, understand:

1 Agentless Architecture

- No daemon on remote server
- Uses SSH
- Push model

Interview question:

How is Ansible different from Puppet or Chef?

Answer:
Agentless vs agent-based.

---

2 Inventory

Inventory defines target machines.

Types:

- Static inventory
- Dynamic inventory

You must understand:

- Groups
- Host variables
- Group variables

Checkpoint:
You should be able to target:

- web servers
- db servers
- all servers

---

3 Ad-hoc Commands

Before playbooks:

```
ansible all -m ping
ansible web -m shell -a "uptime"
```

Understand:
Modules are the core of Ansible.

🏗 PHASE 2 — Playbooks (Core Skill)

Playbook = YAML automation file

Structure:

- hosts
- tasks
- become
- vars

Understand:

- Idempotency (very important)
- Order of execution
- Task failure handling

Interview question:

What is idempotency?

If you can’t explain this, you’re not ready.

🏗️ PHASE 3 — Modules (Important Categories)

You don’t memorize all modules.
You must understand types:

1. Package modules (apt, yum)
2. Service module
3. File module
4. Copy module
5. Template module
6. User module
7. Git module
8. Shell vs Command module (difference!)

Interview favorite:

Difference between shell and command module?

🏗️ PHASE 4 — Variables & Templates

Learn:

- Variables
- Host vars
- Group vars
- Precedence rules

Very important:
Jinja2 templating.

You must know:
How to create a config template and replace variables dynamically.

Example:
Generate nginx.conf dynamically.

This is real DevOps usage.

🏗 PHASE 5 — Roles (Project-Level Structure)

This separates beginner from project-level engineer.

Role structure:

- tasks
- handlers
- templates
- defaults
- vars
- files

Interview question:

Why use roles instead of one big playbook?

Answer:
Reusability, modularity, clean structure.

🏗 PHASE 6 — Handlers

Handlers run only when notified.

Example:
Restart service only if config changes.

You must understand:
Change detection.

🏗️ PHASE 7 — Conditionals & Loops

Learn:

- when conditions
- with_items / loop
- register variable
- debug module

This makes automation intelligent.

🏗️ PHASE 8 — Real Project-Level Tasks

Now simulate real DevOps activities.

Project 1 — Install Docker on Remote Server

Tasks:

- Install dependencies
- Add repo
- Install docker
- Start service
- Enable service
- Add user to docker group

This is classic DevOps task.

Project 2 — Deploy Nginx

- Install nginx
- Copy custom config
- Open firewall
- Restart service

Project 3 — Tomcat Deployment (Your Original Thought)

- Install Java
- Download Tomcat
- Extract
- Deploy WAR
- Create systemd service
- Configure port
- Restart service

Now you understand middleware deployment lifecycle.

Project 4 — Multi-Server Setup

Group:

- web
- db

Deploy:

- MySQL on db
- App on web
- Use variables

Now you simulate real infra separation.

🏗 PHASE 9 — Ansible Vault

Security topic.

Encrypt:

- Passwords
- Secrets

Interview question:

How do you secure sensitive data in Ansible?

Answer:
Ansible Vault.

🏗️ PHASE 10 — Advanced (Interview-Level)

Know conceptually:

- Dynamic inventory (AWS)
- Ansible Galaxy
- Collections
- Strategy plugins
- Parallelism
- Tags
- Check mode
- Dry run
- Ansible configuration file
- SSH key-based authentication

📊 Ansible Competency Model

Beginner:

- Ad-hoc commands
- Basic playbook
- Install package
- Start service

Intermediate:

- Roles
- Templates
- Variables
- Handlers
- Loops
- Conditionals
- Multi-server deployment

Advanced:

- Vault
- Dynamic inventory
- CI integration
- Error handling
- Performance tuning

Senior:

- Large infra automation design
- Role architecture strategy
- Cross-team reusable playbooks
- Governance model

🎯 What DevOps Actually Does with Ansible

Real tasks include:

- Configure new servers
- Patch OS
- Rotate SSH keys
- Deploy middleware
- Update configs across 100 servers
- Restart services in batch
- Bootstrap Kubernetes nodes
- Prepare EC2 for Docker installation

📥 Important Reality

Ansible is powerful.

But in cloud-native companies:

Terraform + Kubernetes dominate.

Ansible is used more in:

- Hybrid infra
- VM-heavy environments
- Enterprise legacy infra

So don’t over-invest time.

🏁 Recommended Timeline for You

You want awareness + interview readiness.

Spend:

2–3 weeks max.

Week 1:
Basics + playbooks + modules

Week 2:
Roles + templates + real projects

Week 3:
Vault + multi-server + interview questions

Then move to Kubernetes.

🔥 Interview Questions You Must Be Ready For

- What is Ansible?
- How is it different from Puppet/Chef?
- What is idempotency?
- What are roles?
- What is Ansible Vault?
- How does inventory work?
- How do you manage secrets?
- How do you structure large playbooks?
- How to make playbooks reusable?
- How to handle failure in Ansible?
- Difference between shell and command module?

🧠 Final Advice for You

Learn Ansible enough to:

- Deploy Tomcat once
- Configure multi-server setup
- Use roles
- Use vault
- Explain architecture

Then move to Kubernetes.

Do NOT get stuck becoming pure configuration engineer.

Your growth path is:

Docker → Kubernetes → Terraform
Ansible = supporting skill

If you want next, I can:

1. Create a 14-day Ansible crash plan
2. Give you a mini real-world project assignment
3. Do mock Ansible interview
4. Compare Ansible vs Terraform clearly

Tell me which one you want.